Assignment 4 (Team-based)

In this assignment, you, as a team, will develop high-quality unit tests, integration tests, test case table, user stories, and acceptance criteria, while also demonstrating proficiency in using GitHub and implementing CI/CD pipelines through GitHub Actions.

1. Activity 1: GitHub/GitHub Actions/Test (17 Points)

Important: The code provided below is intended only as a **starting template**. You may modify, extend, or redesign it as needed, including adding new attributes, methods, constructors, or additional classes. Your implementation should follow sound object-oriented design principles and ensure that all required operations, constraints, and conditions are correctly implemented and thoroughly tested.

The Intelligent Bus Driver Guidance System should include at least the following classes:

1. Driver Class

```
public class Driver {
   private String driverID;
   private String name;
   private int experienceYears;
   private String licenseType; // Light, Medium, Heavy, PublicTransport
   private String address;
   private String birthdate;
}
```

2. Bus Class

```
public class Bus {
   private String busID;
   private int capacity;
   private double fuelLevel;
   private String fuelType; // Diesel, Hybrid, Electricity
}
```

3. DriverRepository Class

```
public class DriverRepositoy {
   // Add (), Update (), Retrieve (), Count () functions
}
```

4. BusRepositoy Class

```
public class BusRepositoy {
   // Add (), Update (), Retrieve (), Count () functions
}
```

Functional Requirements

The system must support the following **operations** for **Drivers**:

Operation	Description
Add	Add new drivers
Retrieve	Retrieve existing drivers
Update	Update existing drivers' details
Count	Return the number of stored drivers

The system must support the following **operations** for **Buses**:

Operation	Description
Add	Add new buses
Retrieve	Retrieve existing buses
Update	Update existing buses' details
Count	Return the number of stored buses

Database

All data must be stored and retrieved using **TXT or JSON files**. You may choose your own:

- TXT/JSON file structure
- storage format
- implementation approach

Note: TXT/JSON files should be human-readable.

Note: A real database system (e.g., MySQL, PostgreSQL, Oracle) is NOT required.

Driver Conditions

The following conditions must be satisfied.

- **D1. Driver ID Rules**
  - driverID must be unique. Duplicate driver IDs are not allowed.
  - The driverID must be exactly 10 characters long
    - The first two characters must be digits between 2 and 9
    - There must be at least two special characters between characters 3 and 8.
    - The last two characters must be uppercase letters (A-Z)
- **D2. Address Format**
  - The driver address must follow the format:
    - Street Number|Street Name|City|State|Country
- **D3. Birthday Format**
  - The birthdate must follow the format: DD-MM-YYYY
- **D4. License Update Restriction**
  - If a driver has more than 10 years of experience, their licenseType cannot be changed during update operations.
- **D5. Immutable Fields**
  - The driverID and name cannot be modified during update operations.

Bus Conditions

- **B1: Bus ID Rules**
  - busID must be unique. Duplicate bus IDs are not allowed.
  - The busID must be exactly 8 characters long
    - All characters must be digits
- **B2: Capacity Update Restriction**
  - busCapacity cannot increase during update operations. However, it can decrease.
- **B3: Driver Age Restriction**
  - Drivers older than 50 years cannot drive buses with a capacity of 50 or more.
- **B4: Electric Bus Restriction**
  - Only drivers with at least 5 years of experience can drive electric buses (fuelType = "Electricity")
- **B5: Driver Licence Restriction**
  - Only drivers holding a Heavy or Public Transport licence are permitted to operate electric and hybrid buses

Activity 1.1 Develop Unit and Integration Testing and Test Case Table (9 Points)

Task 1 – Driver Unit Testing

Write unit tests for the driver-related operations and conditions. **You must write at least 15 meaningful unit test cases for the driver conditions (D1 – D5).** Each condition should have at least **three test cases**.

Your tests should include:

- normal cases
- invalid inputs
- edge cases

Task 2 – Bus Unit Testing

Write unit tests for the bus-related operations and conditions. **You must write at least 15 meaningful unit test cases for the bus conditions (B1 – B5).** Each condition should have at least **three test cases**.

Your tests should include:

- normal cases
- invalid inputs
- edge cases

Task 3 – Driver Integration Testing

Write integration tests for the driver-related operations. **You must write at least 4 meaningful integration test cases.**

Your integration tests should verify:

1. valid drivers are stored correctly
2. invalid drivers are rejected

3. updates are persisted correctly
4. record counts are updated correctly

Note: Integration tests must use real TXT/JSON files and real implementations of your classes

Task 4 – Bus Integration Testing

Write integration tests for the bus-related operations. **You must write at least 4 meaningful integration test cases.**

Your integration tests should verify:

1. valid buses are stored correctly
2. invalid buses are rejected
3. updates are persisted correctly
4. record counts are updated correctly

Note: Integration tests must use real TXT/JSON files and real implementations of your classes

Task 5 – Test Case Documentation

All test cases should be documented in the **Test Case Table** as well. Use **Test Case Template**.

Activity 1.2: Implementation (5 Points)

Create a Java Maven project for the Intelligent Bus Driver Guidance System and implement and thoroughly test the required classes and operations using both unit tests and integration tests according to the specified conditions. You must use **Java** and **JUnit 5**, and you may use **"stubs"** where appropriate.

Activity 1.3: Presentation, GitHub, GitHub Action (3 points)

Create a GitHub repository and upload the implemented classes, associated operations, test cases, and pom.xml file. Configure a GitHub Actions workflow to automatically build and run the tests on every push or pull request.

Record a short demonstration video showing:

1. Running and testing the program locally, including changes made to one or more TXT or JSON files (**1 minute**).
2. Running and testing the program through GitHub Actions (**1 minute**).
3. The GitHub commit history, clearly showing contributions from each team member (**1 minute**).

Note: There is **no requirement** to add your tutor or course coordinator to your GitHub repository.

Recorded Video Requirements

Your recorded video must clearly demonstrate that the project is correctly linked to the remote GitHub repository. There is no need to explain the contents of the **pom.xml** or **.yml files**.

The video should include:

- Pushing code changes to the GitHub repository.
- Triggering the GitHub Actions workflow automatically.
- Running the tests through GitHub Actions.
- Confirmation that all tests pass successfully.

Additional requirements:

- Maximum video length: **THREE minutes**
- Maximum file size: **100 MB**
- Minimum resolution: **720p**

Upload the recorded video to Canvas as part of your submission.

Other Important Notes

Note 1: Your face does not need to be visible in the recorded video.

Note 2: Not all team members are required to speak in the recorded video. One team member may present on behalf of the group, although all members are welcome to participate.

Note 3: You may use one or more TXT/JSON files in your project. The data stored in the file(s) must be human-readable, and you are free to structure the data in any suitable format. There is no requirement to use a database management system such as MySQL, SQL Server, or Oracle.

Note 4: Your Java and JUnit 5 code must include sufficient and meaningful comments to explain the implementation and test cases.

Note 5: You may use any Java IDE, such as Eclipse IDE, Visual Studio Code, or IntelliJ IDEA, to develop and test your project.

Note 6: The project must be developed as a Maven-based Java project. You are not required to implement a graphical user interface (GUI), menu system, or even a main method for user interaction.

Note 7: You are only required to write test cases for the conditions explicitly stated in the specification. However, you are encouraged to include additional relevant test cases if desired.

2. User Story and Acceptance Criteria (8 points)

Based on the requirements collected for the Intelligent Bus Driver Guidance System, each team is asked to write **EIGHT** user stories and **THREE** acceptance criteria for each user story. Each user story and its acceptance criteria have **1 point**. In total, you need to write **8 user stories** and **24 acceptance criteria**. The user stories and acceptance criteria should be written based on the template and principles taught in Week 8. Use the **User Story Template**.